
ARGUS: Cost-Sensitive Fraud Detection under Temporal Validation

A controlled comparison of supervised and unsupervised detectors on 590,540 IEEE-CIS card-not-present transactions, with an explicit monetary decision layer

Yeddula Rudraaksh Reddy^{a,*}, S. N. Chakri^b

^a yeddularudraaksh@gmail.com · LinkedIn: rudraakshreddy · GitHub: rudraakshreddy

^b snchakrim@gmail.com · LinkedIn: snchakri · GitHub: snchakri · snchakri.com

* Corresponding author.

Source repository: <https://github.com/rudraakshreddy/argus>

Deployed application: <https://argus-system.streamlit.app>

Executive Summary

Card-not-present fraud detection is defined by three properties. It is severely *imbalanced* — in the 590,540 IEEE-CIS transactions studied here, 3.50% are fraudulent, so predicting “legitimate” everywhere scores 96.5% accuracy and is worthless. It is *cost-asymmetric*: a missed fraud costs an order of magnitude more than a false alarm, so the conventional 0.5 decision threshold is indefensible. And it is *non-stationary*, because fraud is adversarial and drifts.

The third property dictates the experimental design. Transactions are time-ordered and a deployed model trains on the past to score the future, so ARGUS partitions on transaction time — earliest 80% for training, latest 20% for test — and selects each model’s operating threshold by minimising expected monetary cost rather than by maximising a classification statistic. Four detectors are compared on an identical feature representation of 408 engineered predictors: logistic regression and gradient-boosted trees, against an isolation forest and a deep autoencoder trained on legitimate traffic only.

The gradient-boosted ensemble is best on every measure, reaching AUPRC 0.502 against a prevalence baseline of 0.034, AUROC 0.892, and an expected cost of \$6.99 per transaction against \$29.25 for taking no action. The operating point is where the comparison becomes concrete: it catches 90.2% of fraud while flagging 37.4% of traffic for review, whereas the autoencoder reaches comparable recall only by flagging 70.0% — roughly 38,500 additional manual reviews in this test partition for no additional fraud detected. The two unsupervised detectors are statistically indistinguishable from one another and 5.6× worse than the supervised ensemble, which quantifies the cost of not using available labels.

The largest single effect measured in this study is not a modelling choice. Holding the features and the model configuration fixed and changing only the partition rule from temporal to random inflates AUPRC by 46.1% (0.502 → 0.733) and understates expected cost by 32.8%. A shuffled evaluation would report a system flagging 24.7% of traffic at 13.1% precision; the same system evaluated as it would be deployed flags 37.4% at 8.3% precision.

- The validation protocol dominates: shuffling time-ordered transactions inflates AUPRC by 46%, exceeding the effect of any modelling decision examined here.
- Models are separated by false-positive burden, not by fraud missed. Under a 71:1 cost ratio every detector is driven toward high recall; they differ by nearly 2× in the review workload required.
- Threshold selection for heavy-tailed anomaly scores must be done on the quantile scale. Because AUPRC and AUROC are invariant to monotone transformation, a badly conditioned score leaves them intact while placing the operating point far from optimal — a failure the headline metrics cannot detect.
- Under a realistic review budget the gap widens sharply: at a 1% alert rate the ensemble catches 25.6% of fraud against 0.2–0.4% for the other three, a factor of 60 to 100. Models differ most in the extreme score tail, which is exactly where a capacity-constrained team operates.
- Charging each missed fraud its actual value rather than a flat cost cuts the optimal alert rate from 37.4% to 8.4% — a far more plausible staffing level, obtained without changing the model.
- Week-by-week over the six-week test horizon the ensemble holds AUPRC 0.478–0.540 while the logistic model swings by a factor of three (0.115–0.344), a robustness advantage distinct from its accuracy advantage.

Keywords: fraud detection; class imbalance; cost-sensitive learning; gradient boosting; anomaly detection; temporal validation; concept drift; model explanation.

Contents

1	Introduction	4
1.1	The problem	4
1.2	Contributions	4
1.3	Scope and boundaries	4
2	Related Work	5
2.1	Payment fraud detection	5
2.2	Imbalanced classification and its measurement	5
2.3	Unsupervised anomaly detection	5
2.4	Explanation	5
3	Data and Feature Representation	5
3.1	Dataset	5
3.2	Feature pipeline	6
3.3	Temporal partition	6
4	Models	6
4.1	Logistic regression	6
4.2	Gradient-boosted trees	6
4.3	Isolation forest	6
4.4	Autoencoder	7
4.5	Score-scale conditioning	7
5	Cost-Sensitive Decision Layer	7
6	Results	8
6.1	Discrimination	8
6.2	The cost-optimal operating point	8
6.3	Sensitivity to the cost parameters	9
6.4	Charging each missed fraud its actual value	10
6.5	Detection under a fixed review budget	11
6.6	Are the differences statistically real?	11
6.7	Recalibration	12
6.8	Validation protocol	12
6.9	Stability across the test horizon	12
6.10	Feature attribution	13
7	Discussion	14
7.1	Interpretation	14
7.2	Limitations	15
7.3	Future work	15
7.4	Remarks	15
8	Conclusion	16

Acknowledgements	16
References	18

1 Introduction

1.1 The problem

Card-not-present payment fraud is a detection problem with three properties that jointly determine how it must be modelled and, more importantly, how it must be evaluated. It is severely *imbalanced*: in the data studied here 3.50% of transactions are fraudulent, so a classifier that predicts “legitimate” everywhere attains 96.5% accuracy while providing no value. It is *cost-asymmetric*: a missed fraud costs the median transaction value, while a false alarm costs analyst review time and customer friction — quantities that differ by roughly two orders of magnitude, so the decision threshold cannot be left at the conventional 0.5. And it is *non-stationary*: fraud is adversarial, tactics change, and the statistical relationship between features and label drifts over time [10].

The third property has a direct consequence for experimental design that is frequently overlooked. Transaction data are temporally ordered, and a deployed model is trained on the past and scored on the future. An evaluation that shuffles transactions before splitting them destroys that ordering, allows future information to inform the training set, and produces an accuracy estimate that cannot be realised in deployment. This study adopts a temporal split as its primary protocol and quantifies, on identical features and an identical model, how much optimism a random split introduces.

1.2 Contributions

- (i) A cost-sensitive evaluation framework in which the operating threshold is selected by minimising expected monetary cost rather than by maximising F_1 or defaulting to 0.5, with the resulting threshold, alert rate and confusion matrix reported for every model (Sections 5 and 6).
- (ii) A controlled comparison of two supervised and two unsupervised detectors — logistic regression, gradient-boosted trees, isolation forest and a deep autoencoder — on a common feature representation and a common temporal partition (Section 4).
- (iii) A quantification of the optimism induced by random splitting of temporally ordered transaction data, holding features and model fixed (Section 6.8).
- (iv) An analysis of score-scale conditioning for reconstruction-error detectors, showing why the operating point of an autoencoder must be searched on the quantile scale rather than on the raw error scale (Section 4.5).
- (v) Model explanation via SHAP values [16, 17], calibration assessment, and week-by-week performance tracking across the test period (Section 6).

1.3 Scope and boundaries

Within scope. A single public benchmark, the IEEE-CIS fraud detection dataset: 590,540 card-not-present transactions with 433 predictors spanning 182 days. Binary classification of a transaction as fraudulent at the moment of authorisation. Offline batch evaluation under a temporal partition. A cost-sensitive decision layer parameterised by explicit false-positive and false-negative costs. Interpretability via post-hoc attribution.

Outside scope. *Real-time serving guarantees*: latency, throughput and availability of the scoring service are not benchmarked, and no claim is made about production performance under load. *Adversarial adaptation*: attackers responding to the deployed model are not simulated; the evaluation is on historical data with fixed attacker behaviour. *Graph and network features*: device, IP and card linkage graphs, which are known to carry strong signal in payment fraud, are not constructed — only the tabular fields supplied with the dataset are used. *Sequential and account-level modelling*: each transaction is scored independently; no per-card sequence model or account-lifetime state is maintained. *Online learning and retraining policy*: the cadence at which a deployed model should be refreshed is discussed qualitatively in Section 6.9 but not optimised. *Fairness and regulatory compliance*: disparate impact across protected groups is not assessed, and the dataset’s anonymised fields would not support such an assessment. *Cost parameter estimation*: the false-positive and false-negative costs are treated as given inputs and swept for sensitivity, not estimated from an institution’s own ledger.

2 Related Work

2.1 Payment fraud detection

Dal Pozzolo et al. [5] set out the practitioner’s view of credit-card fraud detection, emphasising that class imbalance, verification latency and concept drift jointly determine what a realistic evaluation looks like; Dal Pozzolo et al. [6] develop that argument into a formal treatment of realistic modelling and propose learning strategies suited to delayed and partial label availability. Whitrow et al. [25] show that aggregating a card’s recent transaction history into features materially improves detection, motivating the card-level aggregate features used here. Bahnsen et al. [1] examine feature engineering strategies specifically for this domain, including periodic encodings of transaction time of the kind adopted in Section 3.

2.2 Imbalanced classification and its measurement

Chawla et al. [3] introduced SMOTE, the most widely used synthetic oversampling method, and a large subsequent literature examines resampling versus cost-sensitive reweighting. Elkan [8] gives the decision-theoretic foundation for the latter: under known misclassification costs, the optimal decision is a threshold on the posterior probability, and altering the training distribution is a means to that end rather than an end in itself. This study therefore uses instance reweighting rather than resampling, and locates the threshold explicitly.

On measurement, Davis and Goadrich [7] and Saito and Rehmsmeier [22] establish that precision–recall curves are more informative than ROC curves under severe imbalance, because the ROC curve’s false-positive-rate axis is dominated by the large negative class and is insensitive to changes that matter operationally. Hand [11] argues more strongly that AUC can be incoherent as a summary when the implied cost distribution differs across classifiers. Fawcett [9] and Provost and Fawcett [21] provide the framework relating ROC geometry to cost-sensitive operating points. Both AUPRC and AUROC are reported here, with AUPRC treated as primary and the cost-optimal operating point treated as the decision-relevant summary.

2.3 Unsupervised anomaly detection

Chandola et al. [2] survey anomaly detection broadly. Liu et al. [15] introduce the isolation forest, which scores anomalies by the expected path length required to isolate a point under random axis-parallel splits and thereby avoids modelling the density of the normal class explicitly. Autoencoder-based detection [12, 23] trains a bottlenecked reconstruction network on normal data and treats reconstruction error as the anomaly score. Both are included here to test a specific claim: that unsupervised detectors are competitive when labels are scarce. Since labels are in fact available in this dataset, the comparison quantifies the cost of not using them.

2.4 Explanation

Lundberg and Lee [16] unify additive feature-attribution methods under the Shapley value, and Lundberg et al. [17] give an exact polynomial-time algorithm for tree ensembles. In fraud detection, attribution serves an operational purpose beyond model debugging: an analyst reviewing a flagged transaction requires a reason, and a regulator may require one.

3 Data and Feature Representation

3.1 Dataset

The IEEE-CIS dataset comprises 590,540 e-commerce transactions with a binary fraud label, of which 3.50% are fraudulent. Transactions span 182 days, indexed by a `timedelta` field from an unspecified reference point. Two tables are joined on the transaction identifier: a transaction table carrying the amount, product code, card attributes, billing address, distances, purchaser and recipient email domains, 14 counting features and 15 `timedelta` features; and an identity table carrying device and network attributes, present for only a subset of transactions. After joining, 434 columns are available, including 339 anonymised engineered features supplied by the dataset authors.

3.2 Feature pipeline

All transforms are fitted on training data only and applied unchanged to the test partition. The pipeline is a fixed sequence of estimators so that this discipline is structural rather than a matter of care.

Amount normalisation. The transaction amount is right-skewed over several orders of magnitude and is log-transformed. It is additionally standardised against the mean and standard deviation of the originating card's own historical amounts, on the reasoning that fraud is often anomalous relative to a card's established pattern rather than in absolute terms.

Cyclical time encoding. The transaction `timedelta` is decomposed into hour-of-day and day-of-week and encoded as sine and cosine pairs, so that the circular topology is respected — hour 23 and hour 0 should be adjacent, which an integer encoding does not express [1].

Missingness indicators. Missingness in this dataset is informative rather than incidental: identity fields are absent for entire transaction classes. Binary indicators are added for every column exceeding a 5% missingness rate, so the pattern of absence remains available after imputation.

Frequency encoding replaces selected high-cardinality categoricals with their observed training frequency, which is a low-variance summary suitable for identifiers with thousands of levels.

Cross-fitted target encoding [18] is applied to a further set of categoricals. The encoding is computed out-of-fold: the value assigned to a training row is derived only from other folds, so the row's own label never enters its own encoding. Without this precaution, target encoding leaks the label directly and inflates apparent performance.

Dimensionality reduction of anonymised features. The 339 anonymised V-features are highly collinear. Those with more than 92% missingness are dropped, and the remainder are reduced by principal component analysis [20] to 30 components retaining 75.7% of variance.

Imputation and scaling. Remaining gaps are filled with training-set medians and all features are standardised, which is required for the logistic and neural models and neutral for the tree ensemble.

The resulting design matrix has 408 numeric features.

3.3 Temporal partition

Transactions are sorted by their time index and the earliest 80% form the training set, the latest 20% the test set. The fraud rate is 3.51% in training and 3.44% in test, so the partition does not introduce a large prevalence shift, but the temporal separation is genuine: no test transaction precedes any training transaction. Section 6.8 contrasts this with the shuffled alternative.

4 Models

4.1 Logistic regression

A regularised linear model on the standardised feature matrix, with class weights inversely proportional to class frequency, implementing the cost-sensitive reweighting of Elkan [8]. It serves as an interpretable reference: any gain reported by a more complex model is measured against it.

4.2 Gradient-boosted trees

An ensemble of 600 depth-8 trees fitted by second-order gradient boosting [4] with learning rate 0.05, row and column subsampling at 0.8, and a minimum child weight of 3. The positive class is weighted by the negative-to-positive ratio, and the training objective is the area under the precision–recall curve, aligning optimisation with the metric of interest under imbalance. Trees are chosen for their capacity to represent the threshold effects and interactions characteristic of fraud rules, and for their tolerance of mixed-scale, partially missing tabular features.

4.3 Isolation forest

An unsupervised ensemble of 200 randomised isolation trees, each built on a subsample of 256 points [15]. Anomaly score is derived from expected isolation path length: points separable in few splits are anomalous. The method uses

no labels at training time.

4.4 Autoencoder

A symmetric feed-forward autoencoder of shape $d \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow d$, with batch normalisation [13], leaky rectified activations and dropout [24] at 0.2 in each hidden block, trained with Adam [14] under mean squared reconstruction loss. Training uses *legitimate transactions only*, so the network learns a compressed representation of normal behaviour; the anomaly score for a test transaction is its reconstruction error under that representation [23].

4.5 Score-scale conditioning

Reconstruction error is heavy-tailed by construction: a small number of extreme outliers produce errors many orders of magnitude larger than the bulk of the distribution. This has no effect on any ranking-based measure — AUROC and AUPRC are invariant to monotone transformation — but it has a decisive effect on threshold selection. Under min–max normalisation to $[0, 1]$, division by an extreme maximum compresses essentially the entire distribution toward zero, so a threshold search on a uniform grid over $[0, 1]$ has almost no resolution in the region where the decision boundary actually lies, and will report a grossly suboptimal operating point while the AUPRC appears unaffected.

Anomaly scores are therefore converted to their empirical cumulative distribution function — a rank transform — before the operating point is selected. Being strictly monotone, this leaves AUROC and AUPRC unchanged, while guaranteeing that the search grid is uniform in probability mass and hence has resolution everywhere. The same principle is applied to the isolation forest scores. This is a small implementation detail with a large consequence, and it is stated explicitly because the failure mode it prevents is silent: the ranking metrics look correct while the deployed decision is badly wrong.

5 Cost-Sensitive Decision Layer

A classifier outputs a score; a fraud system must output a decision. Under known misclassification costs the expected cost per transaction at threshold θ is

$$\mathbb{E}[\text{Cost}](\theta) = \frac{1}{N} \left[c_{\text{FP}} \text{FP}(\theta) + c_{\text{FN}} \text{FN}(\theta) \right], \quad \theta^* = \arg \min_{\theta} \mathbb{E}[\text{Cost}](\theta), \quad (1)$$

where c_{FP} is the cost of investigating a legitimate transaction and c_{FN} the loss from an undetected fraud. The values used are $c_{\text{FP}} = \$12$, approximating thirty minutes of analyst review at prevailing rates, and $c_{\text{FN}} = \$850$. Their ratio, roughly 1:71, is what makes the conventional 0.5 threshold indefensible: at that ratio it is worth accepting many false alarms to prevent one loss.

The value of c_{FN} requires care, because it is a *policy* parameter rather than a statistic of the data. The fraudulent transactions in the test partition have a median value of \$66.40 and a mean of \$150.08, so \$850 is not the transaction value; it is intended to represent the fully loaded consequence of a missed fraud — the value at risk, plus chargeback and scheme fees, dispute handling, and the downstream cost of the compromised credential. Whether \$850 is the right figure is an institution-specific question, and it is not one this study can settle.

Two responses follow, and both are carried through to the results. Section 6 sweeps the cost ratio over three orders of magnitude so the dependence of the operating point on this parameter is stated rather than assumed. And Section 6.4 replaces the flat charge altogether with the *actual transaction value* of each missed fraud, which removes the parameter from the false-negative side of Equation (1) entirely and re-derives every operating point on that basis.

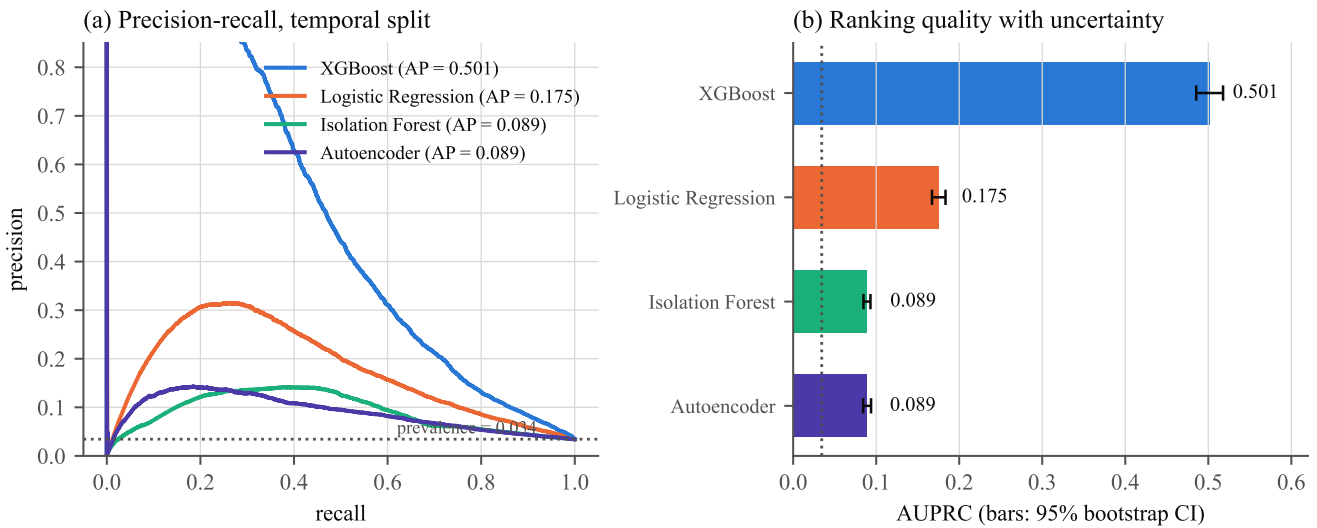
Two points about this formulation deserve emphasis. First, θ^* is a property of the cost structure as much as of the model, so a model comparison at a fixed threshold confounds discrimination with calibration; each model is therefore evaluated at its own cost-optimal threshold. Second, the search over θ is performed on quantiles of each model’s score distribution rather than on a uniform grid, for the reason given in Section 4.5.

Because c_{FP} and c_{FN} are estimates rather than known constants, the cost ratio is swept over three orders of magnitude in Section 6, and the resulting threshold and alert rate are reported as functions of that ratio. This converts an arbitrary-looking parameter choice into a stated sensitivity.

Table 1. Detection performance on the temporal test partition (118,108 transactions, 3.44% fraud). Each model is evaluated at its own cost-optimal threshold. Confidence intervals are 95% bootstrap percentile intervals over 300 resamples.

Model	Type	AUPRC	AUROC	Brier	Cost/txn
XGBoost	Supervised	0.502 [0.486, 0.518]	0.892 [0.887, 0.897]	0.0448	\$6.99
Logistic regression	Supervised	0.175 [0.167, 0.184]	0.828 [0.821, 0.835]	0.1518	\$8.71
Isolation forest	Unsupervised	0.089 [0.085, 0.093]	0.739 [0.729, 0.746]	0.0834	\$11.02
Autoencoder	Unsupervised	0.089 [0.084, 0.094]	0.730 [0.721, 0.739]	0.3180	\$11.07
<i>No model (accept all)</i>	—	0.034	0.500	—	\$29.25

Note. AUPRC for a random classifier equals the prevalence, 0.0344. The “accept all” row gives the cost of taking no action: every fraud is missed at $c_{FN} = \$850$.

**Figure 1.** (a) Precision–recall curves on the temporal test partition; the dotted line is the fraud prevalence, which is the precision a random ranker achieves. (b) AUPRC with 95% bootstrap confidence intervals. The intervals are narrow relative to the differences between models, so the ordering is not in doubt.

6 Results

6.1 Discrimination

Table 1 reports the primary comparison. The gradient-boosted model attains AUPRC 0.502, which is 14.6× the prevalence baseline of 0.034 and 2.9× the logistic model. The two unsupervised detectors are indistinguishable from one another (AUPRC 0.089 each, with overlapping intervals) and are 5.6× worse than the supervised ensemble.

The AUROC column illustrates why it is not used as the primary measure. On AUROC the four models span 0.730 to 0.892 — a range that appears modest and suggests the unsupervised detectors are respectable. On AUPRC the same models span 0.089 to 0.502, a factor of 5.6. The discrepancy is the one identified by Davis and Goadrich [7] and Saito and Rehmsmeier [22]: with 96.6% negatives, the false-positive-rate axis of the ROC curve is insensitive to the large absolute numbers of false alarms that determine whether a system is operationally usable. AUROC compresses precisely the distinction that matters here.

The Brier scores are also informative, and they do not follow the AUPRC ordering. The autoencoder’s 0.3180 is by far the worst, because its score is a rank-transformed reconstruction error which is approximately uniform on $[0, 1]$ and therefore bears no relation to a probability; it ranks acceptably and is calibrated not at all. This separation of ranking quality from probabilistic calibration is why both are reported.

6.2 The cost-optimal operating point

Table 2 makes the practical difference concrete, and it is larger than the AUPRC column alone conveys. All four models achieve broadly similar recall, between 0.866 and 0.902 — because the cost ratio of 71:1 drives every

Table 2. Confusion matrices and operating characteristics at each model’s cost-minimising threshold, $c_{FP} = \$12$ and $c_{FN} = \$850$. The test partition contains 4,064 fraudulent and 114,044 legitimate transactions.

Model	θ^*	TP	FP	FN	Recall	Precision	Alert rate
XGBoost	0.0817	3,664	40,477	400	0.902	0.083	37.4%
Logistic regression	0.3253	3,521	47,249	543	0.866	0.069	43.0%
Isolation forest	0.0637	3,600	75,553	464	0.886	0.046	67.0%
Autoencoder	0.3000	3,641	79,034	423	0.896	0.044	70.0%

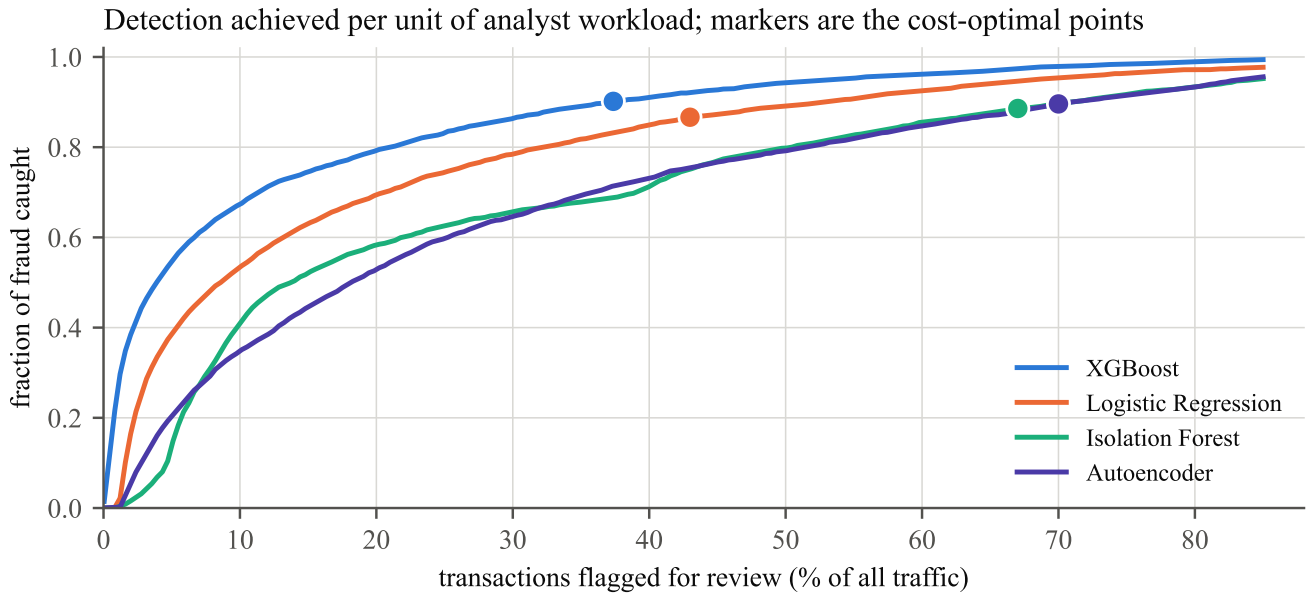


Figure 2. Fraud caught against review workload incurred, traced over the full range of thresholds. Markers give each model’s cost-optimal point. At any fixed workload the supervised ensemble dominates, and the vertical gap between the curves is the fraud that a weaker detector fails to catch for the same analyst effort.

optimiser toward catching almost all fraud. What separates them is the price paid for that recall. The gradient-boosted model reaches 90.2% recall by flagging 37.4% of traffic; the autoencoder reaches a comparable 89.6% recall only by flagging 70.0%. In a portfolio of 118,108 transactions that is a difference of roughly 38,500 additional manual reviews for no additional fraud caught.

Figure 2 presents the same information as a trade-off frontier, which is the form in which the decision is actually made: an institution chooses a review capacity and wants the most fraud detectable within it. At a 20% review budget the gradient-boosted model catches roughly three-quarters of fraud while the unsupervised detectors catch under half.

Figure 3 shows the cost curves. All are convex with interior minima, confirming that neither extreme — reviewing nothing, reviewing everything — is optimal under this cost structure. Against the \$29.25 per transaction incurred by taking no action, the gradient-boosted model reduces expected cost to \$6.99, a 76% reduction; the weakest detector still achieves \$11.07, a 62% reduction. That even a poor detector captures most of the available saving is a direct consequence of the asymmetric cost ratio, and is worth stating plainly: under a 71:1 ratio the dominant term is recall, and models are separated by the false-positive burden rather than by the fraud they miss.

6.3 Sensitivity to the cost parameters

Because c_{FP} and c_{FN} are estimates rather than measured constants, the ratio is swept from 1:1 to 1000:1 in Figure 4. The optimal threshold falls monotonically and the alert rate rises correspondingly, as expected. The practical reading is that the operating point is sensitive to the cost ratio in the region where it is likely to be misestimated, so an institution deploying this system should estimate both costs from its own ledger rather than adopting the values used here. The sweep converts that dependence into a stated relationship rather than a hidden assumption.

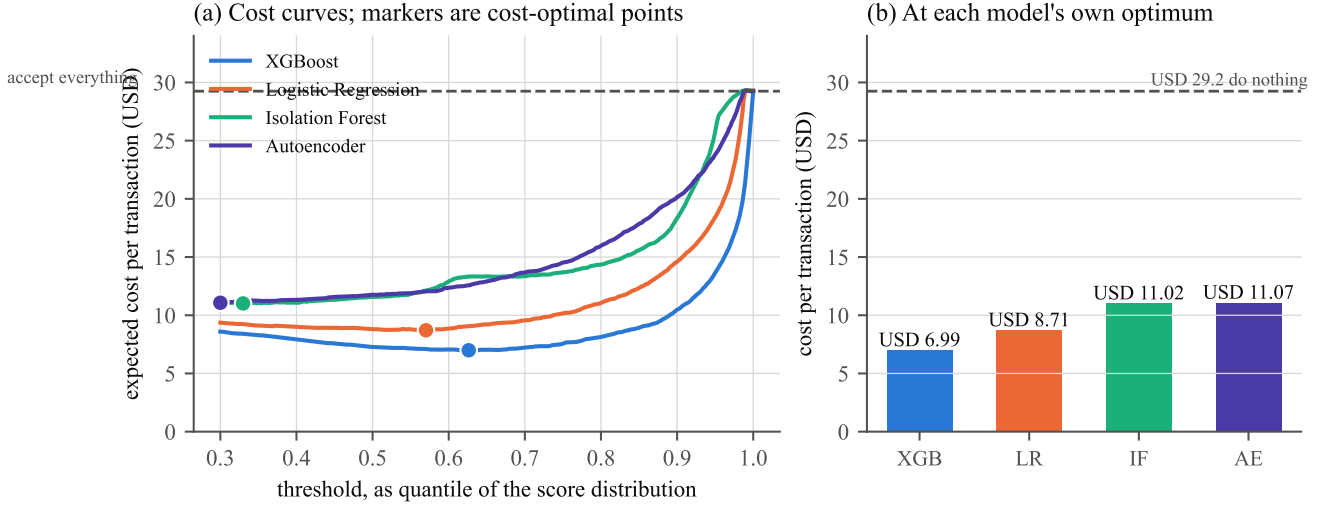


Figure 3. (a) Expected cost per transaction as a function of the decision threshold, expressed as a quantile of each model’s own score distribution; markers give the minima. The dashed line is the cost of taking no action. (b) Minimum achievable cost per transaction by model.

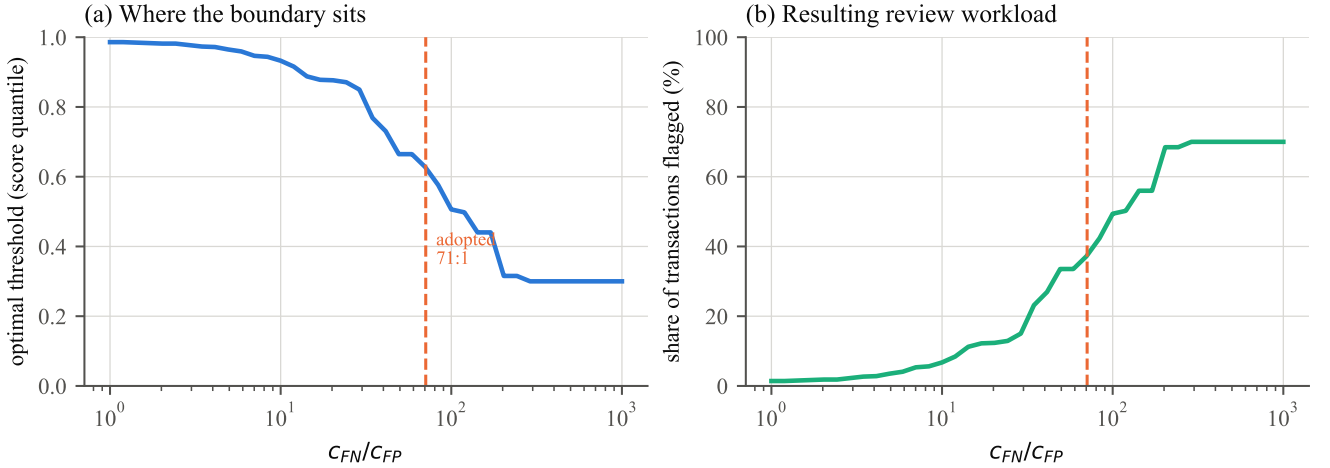


Figure 4. Behaviour of the optimal decision as the cost ratio c_{FN}/c_{FP} is swept over three orders of magnitude. (a) The optimal threshold, expressed as a quantile of the score distribution. (b) The resulting share of traffic flagged for review. The dashed line marks the 71:1 ratio adopted here.

6.4 Charging each missed fraud its actual value

The flat c_{FN} of Table 2 is a modelling convenience. Because the dataset supplies the transaction amount, it can be removed: each missed fraud is charged its own value, so the false-negative term becomes $\sum_{i \in FN} a_i$ where a_i is the transaction amount, and no free parameter remains on that side of the objective.

Table 3 reports the consequence, and it changes the operating point substantially. The implied cost of a missed fraud falls from \$850 to a mean of \$150, so the optimiser becomes far less willing to buy recall with false positives: the gradient-boosted model’s alert rate drops from 37.4% to 8.4%, and expected cost falls to \$2.58 per transaction. This is a much more plausible operating configuration than the flat-cost one — reviewing 8% of traffic is a realistic staffing level in a way that reviewing 37% is not — and it is obtained without changing the model at all.

Two further observations follow from the value columns.

Count-recall and value-recall are nearly identical for the two supervised models (0.645 against 0.644 for the ensemble), which indicates that neither is systematically better at catching large frauds than small ones. For the unsupervised detectors they diverge sharply: the isolation forest catches 44.4% of fraudulent *transactions* but only 24.6% of fraudulent *dollars*. It is preferentially flagging small-value fraud, which is the less valuable half of the problem. A comparison conducted only on count-based recall would have concealed this.

Table 3. Operating points when a missed fraud is charged its own transaction value rather than a flat \$850. False-positive cost remains \$12. “Recall (count)” is the share of fraudulent transactions caught; “recall (value)” is the share of fraudulent dollars.

Model	Cost/txn	Alert rate	Recall (count)	Recall (value)	Value missed
XGBoost	\$2.58	8.4%	0.645	0.644	\$217,206
Logistic regression	\$3.42	15.4%	0.637	0.645	\$216,703
Autoencoder	\$4.68	11.6%	0.378	0.333	\$406,921
Isolation forest	\$5.02	10.9%	0.444	0.246	\$459,995

Note. Fraudulent transactions in the test partition total \$609,934, with a median of \$66.40 and a mean of \$150.08.

Table 4. Share of fraud caught when the alert rate is fixed in advance, as a review team would be staffed. Upper block: share of fraudulent transactions. Lower block: share of fraudulent dollars.

Alert budget	XGBoost	Logistic reg.	Isolation forest	Autoencoder
<i>Recall by transaction count</i>				
0.5%	0.134	0.001	0.000	0.002
1%	0.256	0.004	0.002	0.002
2%	0.387	0.166	0.016	0.055
5%	0.548	0.386	0.139	0.204
10%	0.674	0.534	0.408	0.347
20%	0.793	0.694	0.583	0.528
<i>Recall by fraudulent value</i>				
1%	0.168	0.001	0.004	0.001
5%	0.528	0.337	0.067	0.141
10%	0.670	0.500	0.222	0.289
20%	0.796	0.709	0.363	0.459

The logistic model deserves note here. On flat costs it is clearly second; on amount-weighted costs its value-recall of 0.645 actually edges the ensemble’s 0.644, though it requires nearly twice the alert rate (15.4% against 8.4%) to achieve it. The ensemble’s advantage under this objective is efficiency rather than reach.

6.5 Detection under a fixed review budget

A fraud team is staffed to a capacity, not to a threshold. Table 4 therefore fixes the alert rate and asks how much fraud each model recovers within it — the form in which the question is usually posed operationally.

The result is considerably more decisive than the AUPRC ordering suggests, and it is concentrated at the tight budgets that matter most. At a 1% alert budget the gradient-boosted model catches 25.6% of fraud while the other three catch between 0.2% and 0.4% — a factor of roughly 60 to 100. At 0.5% the gap is starker still: 13.4% against essentially nothing. The advantage narrows steadily as the budget widens, to a factor of 1.14 at a 20% budget.

The interpretation is that the models differ most in the extreme upper tail of their score distributions, which is precisely the region a capacity-constrained review process operates in. A model that ranks the top 1% of transactions well is worth far more than its average-case metrics indicate, and conversely the unsupervised detectors’ respectable AUROC values (0.73–0.74) translate into almost no usable detection at realistic review capacities.

6.6 Are the differences statistically real?

Table 5 tests the ordering rather than assuming it. Every difference involving a supervised model is unambiguous, with intervals far from zero. The single exception is the comparison between the two unsupervised detectors: the difference is 0.0004 with an interval spanning zero and $p = 0.75$, so the isolation forest and the autoencoder are statistically indistinguishable on ranking quality.

That specific null result is worth stating plainly. One method is a randomised tree ensemble with two hyperparameters; the other is a six-layer neural network with batch normalisation, dropout and an Adam training loop. On this problem

Table 5. Paired bootstrap on AUPRC differences, $B = 400$ resamples of the test partition. Each resample scores every model on the same rows, so the comparison is paired.

Comparison	Mean difference	95% CI	p (two-sided)
XGBoost – Logistic regression	+0.326	[0.315, 0.339]	< 0.005
XGBoost – Isolation forest	+0.412	[0.399, 0.428]	< 0.005
XGBoost – Autoencoder	+0.413	[0.400, 0.428]	< 0.005
Logistic reg. – Isolation forest	+0.086	[0.080, 0.093]	< 0.005
Logistic reg. – Autoencoder	+0.087	[0.081, 0.093]	< 0.005
Isolation forest – Autoencoder	+0.0004	[-0.002, 0.003]	0.75

Table 6. Effect of isotonic recalibration, fitted on the first half of the test window and evaluated on the second half.

Model	Brier score		AUPRC	
	Before	After	Before	After
XGBoost	0.0471	0.0247	0.5006	0.4877
Logistic regression	0.1675	0.0434	0.1549	0.1579

they are equivalent, which suggests the limitation is not model capacity but the unsupervised premise itself — that fraud is detectable as a distributional anomaly. As Section 6.5 shows, on this data much of it is not.

6.7 Recalibration

The reliability diagram in Figure 6 showed the logistic model to be badly over-confident, a direct consequence of training with balanced class weights, which deliberately distorts the prior. Table 6 applies isotonic regression [19] to correct it, fitting the calibration map on the first half of the test window and evaluating on the second.

The correction is large and almost free. The logistic model’s Brier score improves by 74% ($0.1675 \rightarrow 0.0434$) and the ensemble’s by 48% ($0.0471 \rightarrow 0.0247$), while AUPRC moves by less than 0.013 in either direction — as expected, since a monotone recalibration cannot materially change the ranking. Where the score is used only to order transactions for review, this step is unnecessary; where it feeds an expected-loss computation of the kind in Section 6.4, it is close to mandatory, and it costs one held-out window and a single monotone fit.

6.8 Validation protocol

Table 7 isolates the effect of the partition rule, with everything else held constant. Shuffling the transaction order inflates AUPRC by 46.1% and understates expected cost by 32.8%. Stated operationally, a random split would report a system flagging 24.7% of traffic at 13.1% precision, whereas the same system evaluated as it would actually be deployed flags 37.4% at 8.3% precision — roughly 50% more manual review work than the shuffled evaluation predicts.

The mechanism is that a random split places transactions from the same period, and frequently from the same card and the same fraud episode, on both sides of the partition. The model therefore encounters test-period conditions during training. Because the deployment scenario is strictly train-on-past, score-future, the temporal figure is the one that estimates realisable performance, and it is the figure reported throughout this study. The gap is not a defect of either number; it is the size of the optimism that a shuffled protocol introduces, and it is large enough to change a deployment decision.

6.9 Stability across the test horizon

Figure 6(a) tracks performance week by week as the model ages without retraining. The gradient-boosted model is stable, ranging from 0.478 to 0.540 with no downward trend over six weeks. The logistic model is not: it ranges from 0.115 to 0.344, a factor of three, and its worst week is worse than the unsupervised detectors’ best. Weekly fraud prevalence itself varies only from 2.84% to 4.14%, so the instability is not explained by a shifting base rate.

Table 7. The same features and the same model configuration under two partitions of the same data.

	Temporal split	Random split	Difference
AUPRC	0.502	0.733	+46.1%
AUROC	0.892	0.951	+6.6%
Brier	0.0448	0.0393	−12.3%
Cost per transaction	\$6.99	\$4.70	−32.8%
Precision at θ^*	0.083	0.131	+58.3%
Alert rate	37.4%	24.7%	−33.9%

Note. Both arms use the identical feature pipeline, identical hyperparameters and the same 80/20 proportion; only the partition rule differs.

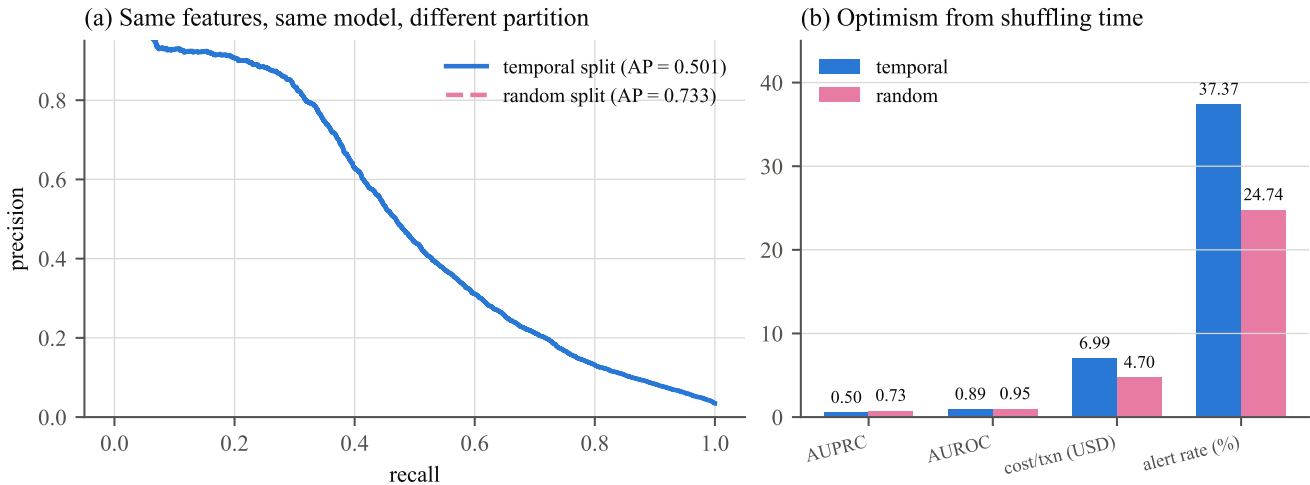


Figure 5. The effect of the partition rule. (a) Precision–recall on the temporal partition, with the random partition’s average precision shown for reference. (b) Metric-by-metric comparison. Every quantity moves in the direction of apparent improvement when time is shuffled.

This is a result about robustness rather than accuracy, and it favours the ensemble for a reason distinct from its higher AUPRC. A linear model’s decision boundary is a single global hyperplane, and a modest shift in the joint feature distribution rotates it relative to the data; a tree ensemble partitions the space locally, so a shift affecting one region leaves the others intact. For a deployed system that will inevitably run for some period between retrains, the stability of the weekly figure matters as much as the average.

Panel (b) reports calibration. The gradient-boosted model tracks the diagonal reasonably across the probability range, while the logistic model — trained with balanced class weights, which deliberately distorts the prior — is systematically over-confident. Its Brier score of 0.1518 against 0.0448 reflects this. Where the output is used only to rank transactions for review, calibration is not required; where a predicted probability feeds a downstream expected-loss calculation, it is, and only the ensemble would be usable without recalibration.

6.10 Feature attribution

Figure 7 reports attribution. Three groups dominate. The counting features (C1, C5, C13, C14) — which encode how many prior transactions share attributes with the current one — are collectively the strongest signal, consistent with the finding of Whitrow et al. [25] that aggregation over a card’s recent history is the most productive feature class in this domain. Amount features follow, and notably the card-relative `amt_zscore` ranks alongside the raw `TransactionAmt`, supporting the design decision in Section 3 to normalise amount against a card’s own history rather than against the population.

The leading anonymised component, `pca_v01`, is the single highest-attribution feature, which is a limitation as much as a result: the strongest individual signal is undocumented, and no domain interpretation of it is possible. The appearance of `M4_missing` confirms that missingness is informative and justifies retaining the indicators.

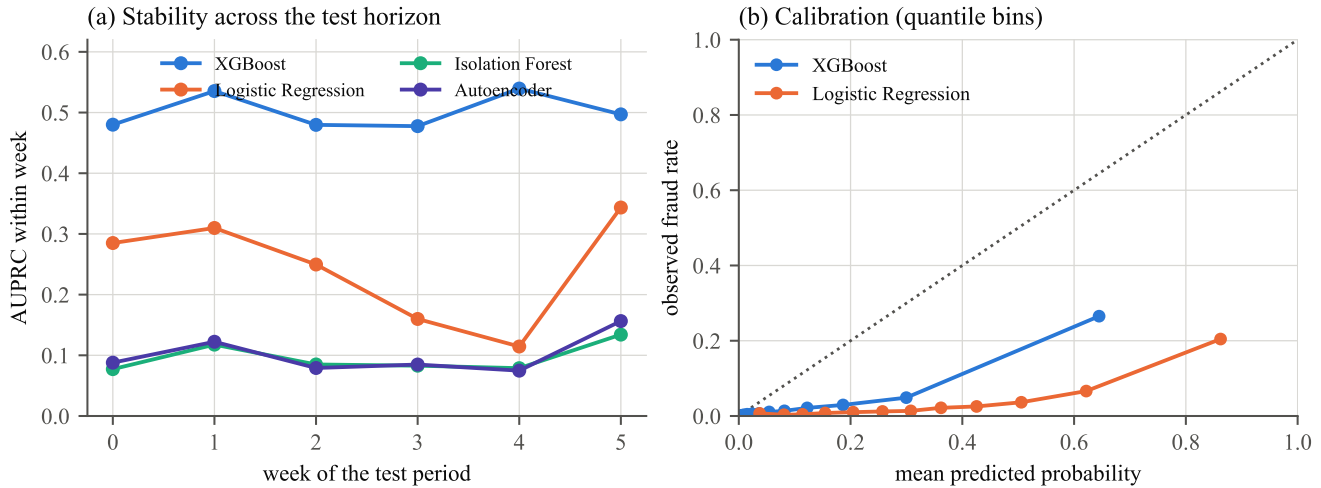


Figure 6. (a) Within-week AUPRC across the six weeks of the test period. (b) Reliability diagram on quantile-spaced bins; the diagonal is perfect calibration.

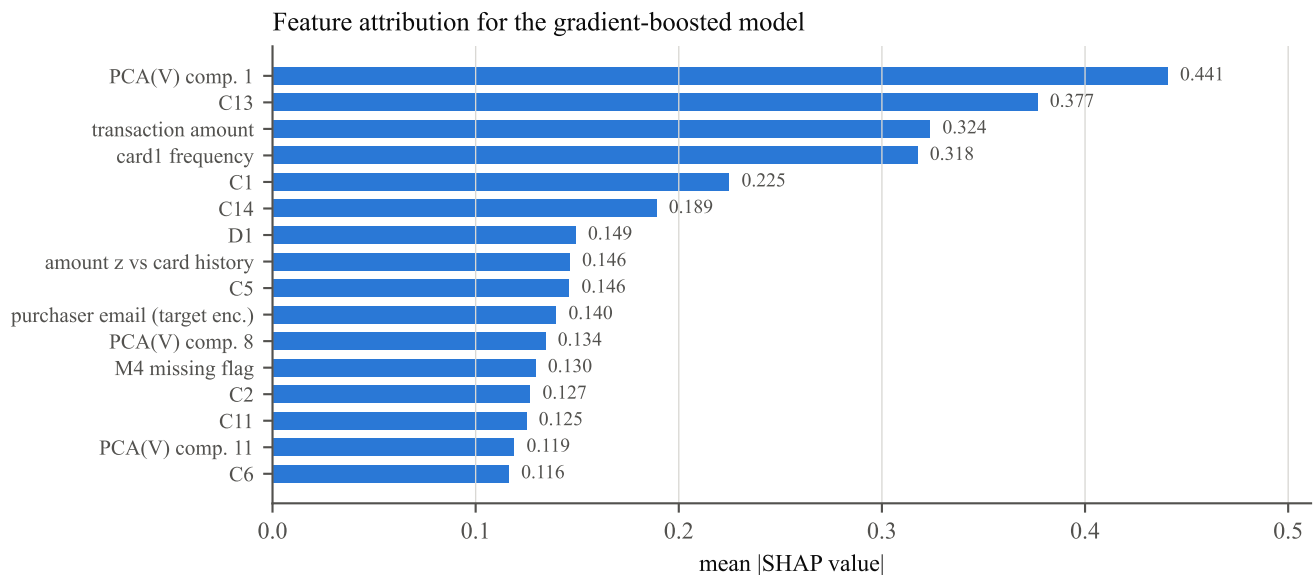


Figure 7. Mean absolute SHAP value over a 4,000-transaction sample of the test partition, for the gradient-boosted model.

7 Discussion

7.1 Interpretation

Four observations generalise beyond this dataset.

The first concerns validation design. Holding features and model fixed and changing only the partition rule moved AUPRC by 46%. No modelling improvement reported in this study is of comparable magnitude. Where a dataset carries a temporal order and the deployment scenario is train-on-past, the partition rule is not a technical detail but the largest single determinant of the reported number.

The second concerns the gap between ranking and deciding. All four detectors reach similar recall at their optimal thresholds; they differ by nearly a factor of two in the workload required to get there. A comparison reported only as AUPRC or AUROC would understate the operational distinction, and one reported at a shared fixed threshold would confound discrimination with calibration. Reporting the confusion matrix at each model’s own cost-optimal point is what makes the comparison decision-relevant.

The third concerns score conditioning, which is a small implementation matter with a disproportionate consequence. Reconstruction error is heavy-tailed, and under min–max normalisation almost the entire distribution collapses toward

zero, leaving a uniform threshold grid with no resolution where the boundary lies. Because AUROC and AUPRC are invariant to monotone transformation, this failure is invisible in the headline metrics while the deployed operating point is badly wrong. The general lesson is that threshold selection must be performed on a scale where the search grid has resolution — quantiles of the empirical score distribution — and that ranking metrics do not validate an operating point.

The fourth concerns the value of labels. Two unsupervised detectors, one classical and one deep, performed almost identically to each other and $5.6\times$ worse than the supervised ensemble on AUPRC. Their common premise is that fraud is statistically anomalous; the result indicates that in this data much fraud is not anomalous in an unsupervised sense — it resembles ordinary traffic on unconditional feature distributions and is separable only through the label. Unsupervised detection remains valuable for the genuinely novel attack that supervised models cannot have learned, but the measured cost of using it as a primary detector is substantial.

7.2 Limitations

Single dataset, 182 days. One merchant population over six months. Drift beyond that window is unobserved, and the six-week stability result should not be extrapolated to longer horizons.

No graph features. Device, IP and card-linkage graphs are the most consequential omission relative to production fraud systems, and their absence probably understates achievable performance materially.

Cost parameters are estimates. $c_{FP} = \$12$ is an approximation and is not varied independently; the sweep in Figure 4 varies only the ratio. The flat $c_{FN} = \$850$ is a policy figure rather than a statistic of the data — the median fraudulent transaction is \$66.40 and the mean \$150.08 — and Section 6.4 shows the choice matters: replacing it with each transaction’s actual value moves the ensemble’s optimal alert rate from 37.4% to 8.4%. Results are reported under both conventions, but an institution would need to estimate the loaded cost of a missed fraud from its own ledger.

Value at risk is not the same as realised loss. Even the amount-weighted analysis charges the full transaction value for a missed fraud, whereas recovery through chargeback reversal is partial and varies by scheme and merchant category. The costs in Table 3 are therefore an upper bound on realised loss.

Label latency ignored. Fraud labels arrive weeks later through chargebacks. This study assumes labels are available at training time, which overstates what a real retraining cycle could use [6].

No hyperparameter search. Configurations were fixed a priori rather than tuned, so the comparison is between reasonable default configurations rather than between each model class at its best.

Anonymised features. 339 predictors are undocumented, and the single strongest feature is one of them, limiting how far the attribution analysis can be interpreted.

7.3 Future work

The most valuable extension is graph features: entity-resolution over device, IP, email and card identifiers, with linkage-derived features such as component size and shared-attribute counts. Second, refining the loss model beyond raw transaction value — incorporating partial chargeback recovery, scheme fees and dispute-handling cost — would sharpen the amount-weighted objective introduced in Section 6.4 into an estimate of realised rather than exposed loss. Third, a realistic retraining study incorporating chargeback label latency — training only on labels that would have been available at each retraining date — would establish the achievable performance under operational constraints. Fourth, extending the temporal evaluation to multiple rolling origins, rather than the single split used here, would allow a Diebold–Mariano-style comparison across periods and would test whether the 46% protocol effect is stable. Finally, the review-budget analysis of Section 6.5 suggests a training objective aligned to it: optimising precision within the top 1% of scores directly, rather than optimising AUPRC and reading the tail off afterwards.

7.4 Remarks

ARGUS is a benchmark study on a public dataset, not a deployed fraud system. Its numbers are conditional on a cost model that an institution would need to re-estimate, and it omits the graph features that production systems depend on.

The finding most worth carrying forward is the one about protocol. The temporal-split result, AUPRC 0.502, is a worse number than the random-split result, AUPRC 0.733, and it is the one this study reports. It would have been straightforward to present the higher figure, since a stratified random split is the default in most tooling and is used without comment in a great deal of published work on this dataset. The difference between the two numbers is not a modelling result — it is the difference between an evaluation that describes deployment and one that does not.

8 Conclusion

ARGUS evaluates four fraud detectors on 590,540 IEEE-CIS card-not-present transactions under a temporal partition, with a decision layer that selects the operating threshold by minimising expected monetary cost rather than by maximising a classification statistic.

The gradient-boosted ensemble is best on every measure: AUPRC 0.502 (against a prevalence baseline of 0.034), AUROC 0.892, Brier 0.0448, and expected cost \$6.99 per transaction against \$29.25 for taking no action. It reaches 90.2% recall while flagging 37.4% of traffic, whereas the autoencoder requires 70.0% of traffic to reach comparable recall — approximately 38,500 additional reviews in this test partition for no additional fraud detected. The two unsupervised detectors are statistically indistinguishable from each other and 5.6× worse than the supervised ensemble on AUPRC, quantifying the cost of not using available labels.

Holding features and model configuration fixed and changing only the partition rule from temporal to random inflates AUPRC by 46.1% and understates expected cost by 32.8%. This exceeds the effect of any modelling choice examined in the study and establishes that, on temporally ordered transaction data, the validation protocol is the dominant determinant of a reported result.

Two further findings are recorded. Week-by-week tracking over the six-week test horizon shows the ensemble stable at AUPRC 0.478–0.540 while the logistic model varies by a factor of three, indicating that the robustness advantage of the ensemble is distinct from its accuracy advantage. And threshold selection for heavy-tailed anomaly scores must be performed on the quantile scale: because ranking metrics are invariant to monotone transformation, a badly conditioned score scale leaves AUPRC and AUROC unchanged while producing an operating point far from the optimum — a failure that the headline metrics cannot detect.

Acknowledgements

The contribution of the secondary author, S. N. Chakri, is acknowledged in respect of the engineering of the feature pipeline and evaluation harness, review of the cost model and validation protocol, and comments on successive drafts. The IEEE-CIS dataset is publicly available under the terms of the competition that released it and is not redistributed by this work. All numerical results and figures were generated by the accompanying source code, available at the repository linked on the title page.

References

- [1] Alejandro Correa Bahnsen, Djamila Aouada, Aleksandar Stojanovic, and Björn Ottersten. Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51:134–142, 2016.
- [2] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.
- [3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [5] Andrea Dal Pozzolo, Olivier Caelen, Yann-Aël Le Borgne, Serge Waterschoot, and Gianluca Bontempi. Learned lessons in credit card fraud detection from a practitioner perspective. *Expert Systems with Applications*, 41(10): 4915–4928, 2014.

- [6] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8):3784–3797, 2018.
- [7] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240, 2006.
- [8] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence*, pages 973–978, 2001.
- [9] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.
- [10] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- [11] David J. Hand. Measuring classifier performance: A coherent alternative to the area under the ROC curve. *Machine Learning*, 77(1):103–123, 2009.
- [12] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [13] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015.
- [14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015.
- [15] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining*, pages 413–422, 2008.
- [16] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, pages 4765–4774, 2017.
- [17] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.
- [18] Daniele Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001.
- [19] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005.
- [20] Karl Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901.
- [21] Foster Provost and Tom Fawcett. Robust classification for imprecise environments. *Machine Learning*, 42(3):203–231, 2001.
- [22] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.
- [23] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2nd Workshop on Machine Learning for Sensory Data Analysis*, pages 4–11, 2014.
- [24] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

- [25] Christopher Whitrow, David J. Hand, Piotr Juszczak, David Weston, and Niall M. Adams. Transaction aggregation as a strategy for credit card fraud detection. *Data Mining and Knowledge Discovery*, 18(1):30–55, 2009.